

```

// api/therapy.js
// Proxy seguro hacia Anthropic — la API key nunca llega al cliente

const ANTHROPIC_URL = 'https://api.anthropic.com/v1/messages';
const MODEL          = 'claude-sonnet-4-20250514';
const MAX_TOKENS     = 1800;

// — Rate limiting en memoria (se resetea con cada cold start) —
const rateMap = new Map();
const RATE_LIMIT    = 10;    // máx peticiones por IP
const RATE_WINDOW    = 60000; // ventana de 60 segundos

function checkRateLimit(ip) {
  const now = Date.now();
  const data = rateMap.get(ip) || { count: 0, start: now };
  if (now - data.start > RATE_WINDOW) {
    rateMap.set(ip, { count: 1, start: now });
    return true;
  }
  if (data.count >= RATE_LIMIT) return false;
  data.count++;
  rateMap.set(ip, data);
  return true;
}

// — Validación de input —
function validateInput({ systemPrompt, userPrompt }) {
  if (!systemPrompt || typeof systemPrompt !== 'string') return 'systemPrompt req
  if (!userPrompt || typeof userPrompt !== 'string') return 'userPrompt requere
  if (systemPrompt.length > 8000) return 'systemPrompt demasiado largo';
  if (userPrompt.length > 4000) return 'userPrompt demasiado largo';
  return null;
}

export default async function handler(req, res) {
  // Solo POST
  if (req.method !== 'POST') {
    return res.status(405).json({ error: 'Método no permitido' });
  }

  // CORS — solo permite tu dominio en producción
  const origin = req.headers.origin || '';
  res.setHeader('Access-Control-Allow-Origin', origin);
  res.setHeader('Access-Control-Allow-Methods', 'POST');

```

```

res.setHeader('Access-Control-Allow-Headers', 'Content-Type');

// Rate limiting
const ip = req.headers['x-forwarded-for']?.split(',')[0] || req.socket.remoteAd
if (!checkRateLimit(ip)) {
  console.warn(`[therapy] Rate limit alcanzado - IP: ${ip}`);
  return res.status(429).json({ error: 'Demasiadas peticiones. Espera un moment
}

// Parsear body
let body;
try {
  body = typeof req.body === 'string' ? JSON.parse(req.body) : req.body;
} catch {
  return res.status(400).json({ error: 'JSON inválido' });
}

// Validar
const validationError = validateInput(body);
if (validationError) {
  return res.status(400).json({ error: validationError });
}

const { systemPrompt, userPrompt } = body;

// API key desde variable de entorno - nunca hardcodeada
const apiKey = process.env.ANTHROPIC_API_KEY;
if (!apiKey) {
  console.error(`[therapy] ANTHROPIC_API_KEY no configurada`);
  return res.status(500).json({ error: 'Configuración del servidor incompleta'
}

console.log(`[therapy] Petición - IP: ${ip} - ${new Date().toISOString()}`);

try {
  // Llamada a Anthropic con streaming
  const anthropicRes = await fetch(ANTHROPIC_URL, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'x-api-key': apiKey,
      'anthropic-version': '2023-06-01',
    },
    body: JSON.stringify({
      model: MODEL,
      max_tokens: MAX_TOKENS,
      stream: true,

```

```

        system:    systemPrompt,
        messages: [{ role: 'user', content: userPrompt }],
    }},
    });

    if (!anthropicRes.ok) {
        const errText = await anthropicRes.text();
        console.error(`[therapy] Error Anthropic ${anthropicRes.status}: ${errText}`);
        return res.status(502).json({ error: 'Error del servicio de IA' });
    }

    // Reenviar el stream de SSE directamente al cliente
    res.setHeader('Content-Type', 'text/event-stream');
    res.setHeader('Cache-Control', 'no-cache');
    res.setHeader('Connection', 'keep-alive');

    const reader = anthropicRes.body.getReader();
    const decoder = new TextDecoder();

    while (true) {
        const { done, value } = await reader.read();
        if (done) break;
        const chunk = decoder.decode(value, { stream: true });
        res.write(chunk);
    }

    res.end();
    console.log(`[therapy] Stream completado - IP: ${ip}`);

} catch (err) {
    console.error('[therapy] Error interno:', err.message);
    if (!res.headersSent) {
        res.status(500).json({ error: 'Error interno del servidor' });
    } else {
        res.end();
    }
}
}

```